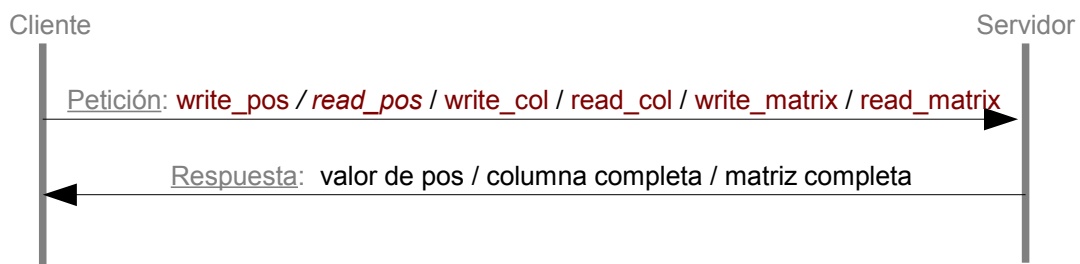


PRÁCTICA 2: Opción 1

La práctica consistirá en la realización de sistema cliente-servidor formado por dos aplicaciones que se comunican mediante por el puerto RS-232 (se utilizará la aplicación VSPE para emular los puertos RS-232) utilizando la librería explicada en clase. El servidor tendrá **una matriz de *fil*col enteros*** (usar directiva *#define*) y ofrecerá al cliente los siguientes servicios:

- *write_pos()*: Escribe una posición de la matriz con los datos enviados desde el cliente.
- *read_pos()*: Lee una posición de la matriz y envía el valor al cliente.
- *write_col()*: Escribe una columna de la matriz con los datos enviados desde el cliente.
- *read_col()*: Lee una columna de la matriz y se la envía al cliente.
- *write_matrix()*: Escribe la matriz completa con los datos enviados desde el cliente.
- *read_matrix()*: Lee la matriz completa y se la envía al cliente.

Para dar la órdenes al proceso del servidor se implementará un cliente que enviará las ordenes a través de un puerto RS-232 virtual (usar el *software VSPE*) y que permitirá probar cada una de las órdenes que oferta el servidor y mostrará la información recibida. El esquema de la comunicación de la siguiente figura según el caso:



Esquema de mensajería

Las dos aplicaciones implementadas, así como las funciones creadas, deben estar diseñadas correctamente de acuerdo a los principios de diseño estudiados en clase. Se deja libertad en la implementación de los mensajes, con la restricción de que cada orden sólo se puede enviar **un único mensaje que debe contener toda la información necesaria para el proceso que lo recibe**. Ésta es la funcionalidad mínima de la práctica, el alumno puede completarla con cualquier cosa que se le ocurra (siempre con criterio).

PRÁCTICA 2: Opción 2

La práctica consistirá en la realización de sistema cliente-servidor formada por dos aplicaciones que se comunican mediante por el puerto RS-232 (se utilizará la aplicación VSPE para emular los puertos RS-232) utilizando la librería explicada en clase. El servidor tendrá un **vector de tamaño TAM múltiplo de N bytes** (usar para estos dos la directiva *#define*) y ofrecerá al cliente los siguientes servicios:

- **setBlock()**: Escribe un bloque de tamaño *N* en una posición concreta direccionada como bloque. Cada bloque es un conjunto de bytes consecutivos.
- **getBlock()**: Obtiene un bloque de tamaño *N* en una posición concreta direccionada como bloque y lo envía al cliente.
- **setConsBlocks()**: Escribe *M* bloques consecutivos de tamaño *N* a partir de una posición concreta direccionada como bloque. Si *M* es mayor que los que se pueden insertar sólo inserta los que se puedan.
- **getConsblocks()**: Lee *M* bloques consecutivos de tamaño *N* a partir de una posición concreta direccionada como bloque y se los envía al cliente.
- **setVect()**: Escribe el vector completo con la información enviada desde el cliente.
- **getVect()**: Lee el vector completo y se lo envía al cliente.

Para dar la órdenes al proceso del servidor se implementará un cliente que enviará las ordenes a través de un puerto RS-232 virtual (usar el *software VSPE*) y que permitirá probar cada una de las órdenes que oferta el servidor y mostrará la información recibida. El esquema de la comunicación de la siguiente figura según el caso:



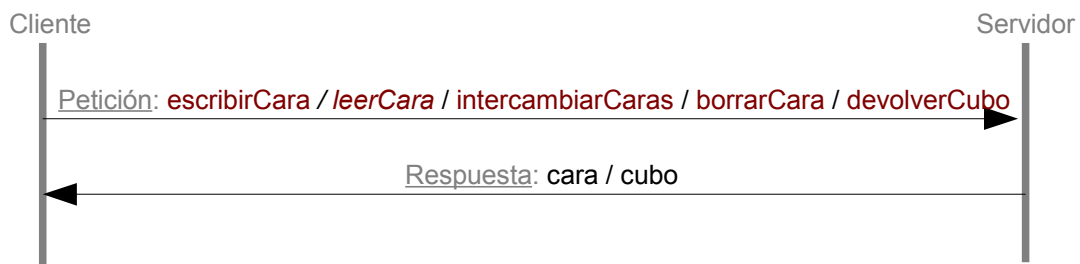
Las dos aplicaciones implementadas, así como las funciones creadas, deben estar diseñadas correctamente de acuerdo a los principios de diseño estudiados en clase. Se deja libertad en la implementación de los mensajes, con la restricción de que cada orden sólo se puede enviar **un único mensaje que debe contener toda la información necesaria para el proceso que lo recibe**. Ésta es la funcionalidad mínima de la práctica, el alumno puede completarla con cualquier cosa que se le ocurra (siempre con criterio).

PRÁCTICA 2: Opción 3

La práctica consistirá en la realización de sistema cliente-servidor formada por dos aplicaciones que se comunican mediante por el puerto RS-232 (se utilizará la aplicación VSPE para emular los puertos RS-232) utilizando la librería explicada en clase. El servidor tendrá una estructura de datos a elegir por el alumno que representará las 6 caras de $N*N$ de un cubo *con los valores 0 o 1* en cada posición del cubo (usar la directiva *#define* para N) y ofrecerá al cliente los siguientes servicios:

- *escribirCara()*: Escribe una cara del cubo con los valores recibidos desde el cliente.
- *leerCara()*: Lee los valores de una cara del cubo y los envía al cliente.
- *intercambiarCaras()*: Intercambia los valores de dos caras del cubo indicadas por el cliente.
- *borrarCara()*: Rellena una cara indicada del cubo con ceros.
- *devolverCubo()*: Devuelve todos los valores de todas las caras del cubo al cliente.

Para dar la órdenes al proceso del servidor se implementará un cliente que enviará las ordenes a través de un puerto RS-232 virtual (usar el *software VSPE*) y que permitirá probar cada una de las órdenes que oferta el servidor y mostrará la información recibida. El esquema de la comunicación de la siguiente figura según el caso:



Esquema de mensajería

Las dos aplicaciones implementadas, así como las funciones creadas, deben estar diseñadas correctamente de acuerdo a los principios de diseño estudiados en clase (uso de módulos y funciones). Se deja libertad en la implementación de los mensajes, con la restricción de que cada orden sólo se puede enviar **un único mensaje que debe contener toda la información necesaria para el proceso que lo recibe**. Ésta es la funcionalidad mínima de la práctica, el alumno puede completarla con cualquier cosa que se le ocurra (siempre con criterio).

PRÁCTICA 2: Opción 4

La práctica consistirá en la realización de sistema cliente-servidor formado por dos aplicaciones que se comunican mediante por el puerto RS-232 (se utilizará la aplicación VSPE para emular los puertos RS-232) utilizando la librería explicada en clase. El servidor tendrá una variable con la estructura de C llamada *struct funcion* que tiene la siguiente forma:

```
struct funcion {  
    int tamI;  
    BYTE *input;  
    int tamO;  
    BYTE *output;  
};
```

Los campos de la estructura *tamI* e *input* se usan para que el servidor tenga información relativa a un vector que se recibirá del cliente. Los campos *tamO* u *output* contienen la información relativa a un vector generado mediante un método que se indica a continuación.

El método trabajará de la siguiente forma sobre cada uno de los valores del vector *input* según los dos primeros bits del valor hará la siguiente función:

- 00: Tomará de dos en dos bits consecutivos y los transformará a un único bit según la siguiente tabla (función XOR).

Bits entrada	Bit salida
0 0	0
0 1	1
1 0	1
1 1	0

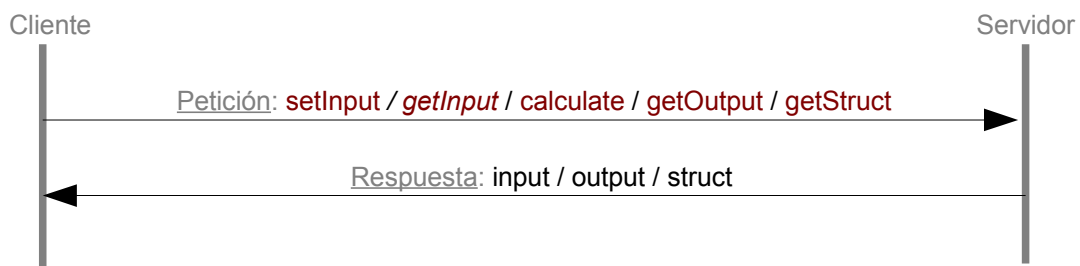
- 01: Invertirá los bits de la tabla, es decir, los bits 0 y 7 se intercambian entre sí, los bits 1 y 6 se intercambian entre sí, y así sucesivamente.
- 10: Cada bit es negado, es decir, un bit a 0 pasa a ser 1 y viceversa.
- 11: Cada bit se convierte en dos mediante la siguiente tabla:

Bit entrada	Bits salida
0	0 1
1	1 0

De esta forma, el tamaño del vector *output* cambiará dado que los casos 00 y 11 convierten dos bytes en uno o uno en dos respectivamente. El servidor ofrecerá al cliente los siguientes servicios:

- *setInput()*: Escribe los campos *tamI* e *input* con los valores recibidos desde el cliente.
- *getInput()*: Lee los campos *tamI* e *input* y los envía al cliente.
- *calculate()*: Calcula *output* basándose en el contenido del campo *input* usando el método explicado arriba.
- *getOutput()*: Lee los campos *tamO* e *output* y los envía al cliente.
- *getStruct()*: Envía la estructura completa al cliente.

Para dar la órdenes al proceso del servidor se implementará un cliente que enviará las ordenes a través de un puerto RS-232 virtual (usar el *software VSPE*) y que permitirá probar cada una de las órdenes que oferta el servidor y mostrará la información recibida. El esquema de la comunicación de la siguiente figura según el caso:



Esquema de mensajería

Las dos aplicaciones implementadas, así como las funciones creadas, deben estar diseñadas correctamente de acuerdo a los principios de diseño estudiados en clase (uso de módulos y funciones). Se deja libertad en la implementación de los mensajes, con la restricción de que cada orden sólo se puede enviar **un único mensaje que debe contener toda la información necesaria para el proceso que lo recibe**. Ésta es la funcionalidad mínima de la práctica, el alumno puede completarla con cualquier cosa que se le ocurra (siempre con criterio).